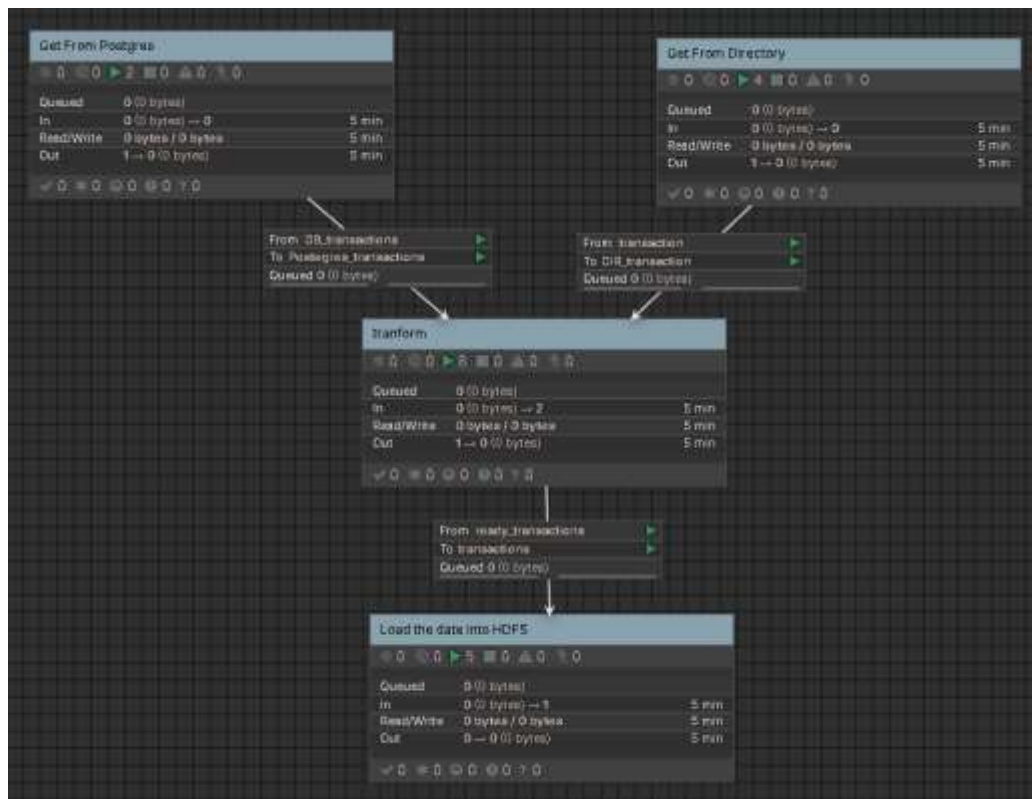


Architecture Diagram



I divide the project into those four process groups and connect them with input and output ports to keep the flow organized and scalable.

Get from Postgres group to pull data from the database.

Get from Directory group to read files from a folder.

Transformer group to clean and prepare the data.

Load the data into HDFS group to store the processed data in Hadoop Distributed File System.

Ask me Why?

Because structuring the flow this way, I make it easier to maintain, extend, and scale later and it's common best practise .

I will explain each process group and the decisions down there ,,,

Real-Time Data Simulation

I created python script to generate duplicate messy json files every 5 sec that simulate real world Transactions and also save it in directory

```
#!/usr/bin/env python3
import time
import random
import os
from datetime import datetime

# Where our messy files will be saved at all! and pick them up
output_dir = "Lab_data"

# Create the parent if it doesn't exist
os.makedirs(output_dir, exist_ok=True)

# Possible transaction statuses
STATUSES = ["COMPLETED", "PENDING", "FAILED", "REJECTED"]

def generate_record():
    """Generate a single pseudo-random transaction record as a dict"""
    current_dt = datetime.now()

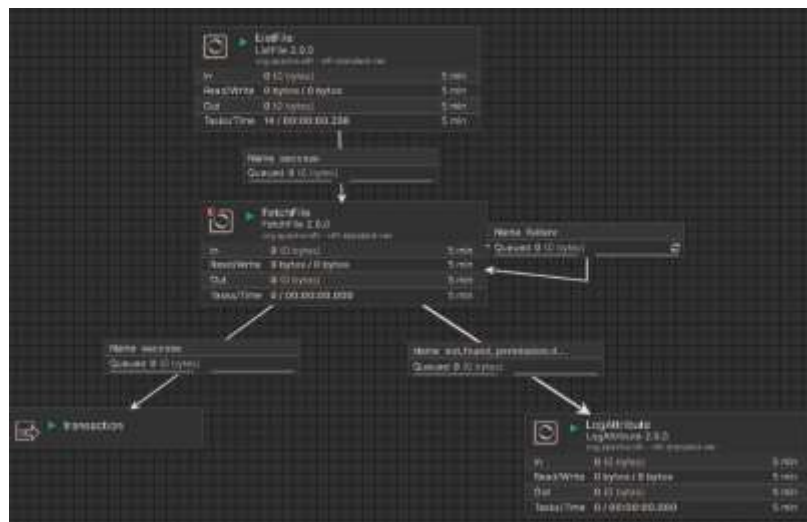
    # Create the ID for our transaction (transaction_id, json key)
    dt_string = current_dt.strftime("%Y-%m-%d-%H-%M-%S")
    transaction_id = f"transaction_{dt_string}"

    customer_id = random.randint(1000, 9999)
    amount = round(random.uniform(10.0, 100.0), 2)
    status = random.choice(STATUSES)
    data_str = current_dt.strftime("%Y-%m-%d-%H-%M-%S")
```

The script in the same folder out there

File-Based Ingestion using NiFi

This is the first process group **Get from Directory**



List file processor: I used it as observer when there are any new Transactions till the next process to fetch it and it runs every 5 sec .

FetchFile processor: I used it as hunter when the observer tell him there is file then he fetch it and it runs every time.

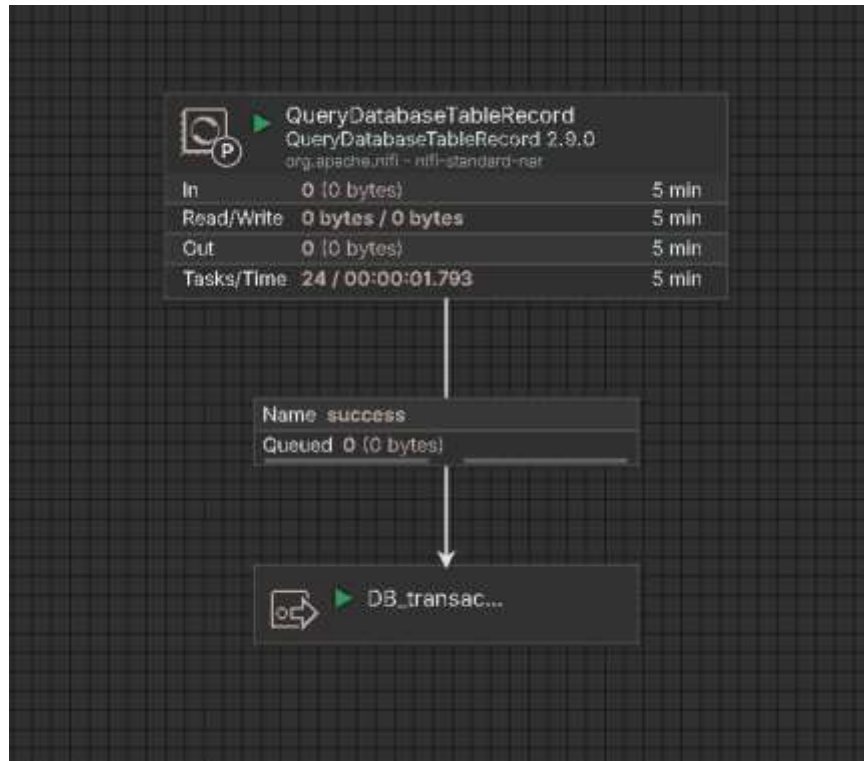
Also **Log attribute** to log **not found** and **permission denied** files ,,

but for **failure** i make it retry after penalized duration ,

for **success** files its go to the next layer

Database Ingestion

This is the second process group **Get from Postgres**



in this group I used the **QueryDatabaseTableRecord** to get the data from postgres database every 5 sec because server re resources

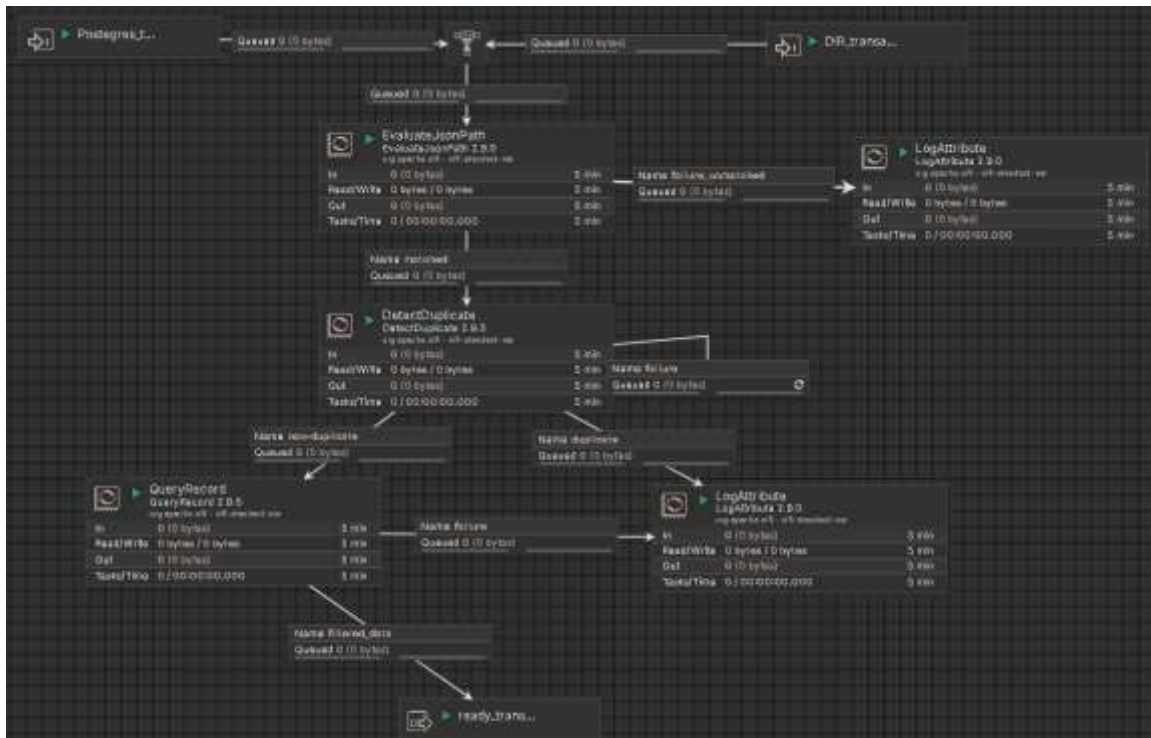
ask me why **QueryDatabaseTableRecord** but not **QueryDatabaseTable** or **ExecuteSQL**?

The main Big reason is :its directly converts database rows into json or any data type which that what I want the most beside That I can set format schema in JsonRecordSetWriter and because the coulmnns names in DB not the same as the json files and to simplify the transformer layer which is the next

Other reason : I want it fetch only the new records which It can handle this part effectively .

Data Transformations

This is the third process group **Transformer**



Because of the **JsonRecordSetWriter** and when I use it in the **QueryDatabaseTableRecord** the data come from the two source with the same schema , so now I can use the same flow for all data whatever the source is ,

The data come from two input ports from directory and pstegres I set the Prioriti for the older records

and I set the **Back Pressure** on 3000 flowfile to handle the peak hours

the Run Schedule :this layer or process group every time is runing because its important that not stop working

EvaluateJsonPath : I used it to read the flowfile and get the transaction_id as Aturbute because I need it in next process

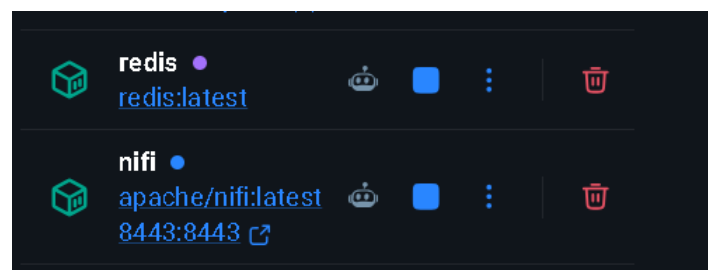
DetectDuplicate : i used this process to detect the duplicated transactions because that cuase confilct in the data

But why I chose this process not others ?

I chose DetectDuplicate processor to prevents any record with the same ID from passing through more than once.the main feture is I link it to **Redis** as an external DistributedMapCache to sync across multiple NiFi nodes simultaneously in future which ensure integrity for my data

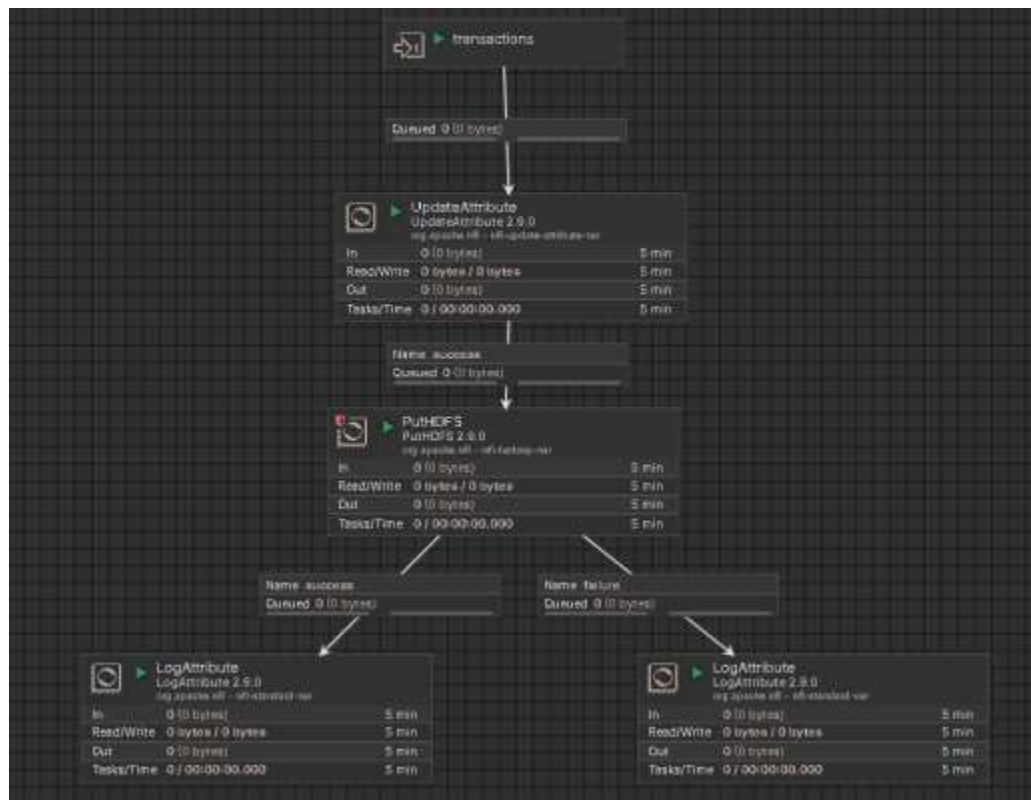
QuerRecord : simply I used it to ensure there is any vaule is null in record .

logAttribute : for log all failurs , dublicated and unmatched transactions



Load to HDFS

This is the third process group **Load the data into HDFS**



I get the transactions from the previous group process (transformer) as clean non-duplicates trans

So now I have to store them in Hadoop file system

Update Attribute : I used it to create attribute contain the file name actually the process generate it

Put HDFS : to connect the hdfs server and save the transactions distributed in file system

logAttribute : to log all the status

now the data is ready

Browse Directory

Authentication

Go!



Show 25 entries

Search:

☐	Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name	☐
☐	-rw-r--r--	nifi	nifi	168 B	May 07 23:11	1	128 MB	Transaction_20260507_231107_813540.json	
☐	-rw-r--r--	nifi	nifi	168 B	May 07 23:11	1	128 MB	Transaction_20260507_231113_361705.json	
☐	-rw-r--r--	nifi	nifi	169 B	May 07 23:11	1	128 MB	Transaction_20260507_231119_361733.json	
☐	-rw-r--r--	nifi	nifi	166 B	May 07 23:11	1	128 MB	Transaction_20260507_231124_362630.json	
☐	-rw-r--r--	nifi	nifi	167 B	May 07 23:11	1	128 MB	Transaction_20260507_231129_364396.json	

Sample data before and after transformation

Before

```
[
  {
    "Transaction_ID": "Transaction_20260507202822747162",
    "Customer_ID": null,
    "Amount": null,
    "Status": "REFUNDED",
    "Date": "2026-05-07 20:28:22"
  },
  {
    "Transaction_ID": "Transaction_1001",
    "Customer_ID": 88,
    "Amount": 250.00,
    "Status": "SUCCESS",
    "Date": "2026-05-08 12:00:00"
  },
  {
    "Transaction_ID": "Transaction_1001",
    "Customer_ID": 88,
    "Amount": 250.00,
    "Status": "SUCCESS",
    "Date": "2026-05-08 12:00:00"
  }
]
```

after

```
}
  "Date": "2026-05-07 20:28:22",
  "Status": "REFUNDED",
  "Amount": null,
  "Customer_ID": null,
  "Transaction_ID": "Transaction_20260507202822747162",
  }
```